

MTH401 HW2 Report

Sena ERSOY-040200434

18 April 2026

1 RISC-V Custom ISA Tek Çevrimlik (Single-Cycle) İşlemci Mimarisi ve Akışı

1.1 Tek Çevrimlik (Single-Cycle) İşlemci Nedir?

Tek çevrimlik işlemci, bir bilgisayarın en temel yapıtaşıdır. Adından da anlaşılacağı üzere, bu mimaride bir komutun getirilmesi, şifresinin çözülmesi, hesaplamasının yapılması ve sonucun kaydedilmesi işlemlerinin **tamamı tek bir saat vuruşunda (clock cycle)** gerçekleşir. Saat sinyali her yükseldiğinde (**posedge clk**), işlemci yepyeni bir komuta geçer. Bu yapı, anlaşılması ve tasarlanması en kolay mimari olsa da, en yavaş komutun süresi tüm saat frekansını belirlediği için modern sistemlerde yerini pipeline (boru hattı) mimarilerine bırakmıştır.

Ancak bu ödevde geliştirdiğimiz işlemci sıradan bir RISC-V değil; opcode bitlerinin yerlerinin değiştirildiği, bize özel şifrelenmiş bir **Custom ISA** (Özel Komut Seti Mimarisi) işlemcisidir.

1.2 İşlemcinin Modüler Yapısı ve Görevleri

Sistemimiz, `riscv_singlecycle.sv` (Top Module) içinde birleştirilmiş çeşitli donanım bloklarından oluşur. Her bir bloğun görevi şöyledir:

- **Program Counter (PC - Program Sayacı):** İşlemcinin o an hangi komutu okuyacağını gösteren 32 bitlik adrestir. Sistem resetlendiğinde, PC değeri RISC-V standart başlangıç adresi olan `0x8000_0000` değerine çekilir.
- **Instruction Memory (IMEM - Komut Belleği):** PC'den gelen 32 bitlik adresi alır ve o adreste yazan 32 bitlik şifreli (Custom ISA) komutu dışarı verir.
- **Decoder (Şifre Çözücü ve Kontrol Birimi):** İşlemcinin beynidir. Standart RISC-V'den farklı olarak opcode bitlerini [19:13] aralığından çeker. Hangi registerların okunacağını, ALU'nun ne işlem yapacağını ve belleğe yazılıp yazılmayacağını belirten kontrol sinyallerini üretir.
- **Register File (register Öbeği):** İçinde 32 adet 32-bitlik kutucuk bulunur. `x0` registersı donanımsal olarak sıfıra sabitlenmiştir. ALU veya Data Memory'den gelen sonuç, hedef registerya (`rd`) yazılır.
- **ALU (Aritmetik Mantık Birimi):** İki giriş alır ve Decoder'ın verdiği emre göre toplama, çıkarma, kaydırma veya mantıksal işlemler yapar.
- **Data Memory (DMEM - Veri Belleği):** Sadece Load (`LW`) ve Store (`SW`) komutlarında aktif olur.

1.3 Bir Komutun Anatomisi: `addi x2, x0, 5` Komutunun Satır Satır Yolculuğu

Sistemimizin nasıl çalıştığını kavramak için, simülasyonda başarıyla çalıştırdığımız `addi x2, x0, 5` komutunun tek bir saat vuruşundaki donanımsal yolculuğunu inceleyelim. Test dosyamızda bu komut Custom ISA şifreleyicimiz tarafından `0x005c8002` Hex değeriyle belleğe yazılmıştır.

Bu komutun 32 bitlik şifre dağılımı aşağıdaki gibidir:

Bit Aralığı	Değer (Binary)	Custom ISA Formatı	Anlamı
[31:20]	000000000101	imm12 (Sabit Sayı)	İşleme girecek olan 5 değeri.
[19:13]	1100100	opcode (İşlem Kodu)	0x64 : Custom I-Type ALU komutu.
[12:8]	00000	rs1 (Kaynak register 1)	x0 registersının adresi.
[7:5]	000	funct3 (Fonksiyon Tipi)	Toplama (ADD) işlemi.
[4:0]	00010	rd (Hedef register)	Sonucun yazılacağı x2 adresi.

1.4 Kod Seviyesinde Akış: Sinyallerin Donanımdaki Yolculuğu

Adım 1: PC ve Komutu Okuma

Her şey `riscv_singlecycle.sv` içinde başlar. İşlemci `0x8000_0000` adresini Instruction Memory'ye (IMEM) bağlar.

```
1 // RAM word (32-bit) tabanlı olduğu için adresin son 2 biti atılır.
2 assign instr_ = imem_data[pc_reg[12:2]];
```

Listing 1: Instruction Fetch (`riscv_singlecycle.sv`)

Adım 2: Decoder (Şifre Çözücü)

`instr_` kablosundaki 32 bitlik şifreli değer, `decoder.sv`'nin girişine ulaşır ve Custom ISA kurallarına göre dilimlenir.

```
1 assign opcode = instr[19:13]; // Sonuc: 7'h64
2 assign rs1    = instr[12:8];  // Sonuc: 5'd0 (x0)
3 assign rd     = instr[4:0];   // Sonuc: 5'd2 (x2)
4 assign imm    = instr[31:20]; // Sonuc: 12'd5
5
6 always_comb begin
7     case(opcode)
8         7'h64: begin // Custom I-Type ALU komutu!
9             update_o    = 1; // Gecerli komut
10            reg_write_en = 1; // rd'ye yazma izni
11            alu_ctrl     = 3'b000; // ALU: Toplama yap
12        end
13    endcase
14 end
```

Listing 2: Komut Parcalama ve Kontrol Sinyalleri (`decoder.sv`)

Adım 3: register Okuma ve Hesaplama (Execute)

Register file `x0`'ın değerini okur ve ALU işlemi gerçekleştirir.

```
1 always_comb begin
2     case(alu_ctrl)
3         3'b000: alu_result = in1 + in2; // 0 + 5 islemi gerceklesir
4     endcase
5 end
```

Listing 3: ALU İşlemi (`alu.sv`)

Adım 4: Geri Yazma (Write-back) ve PC Güncelleme

ALU'dan çıkan **5** sonucu, `posedge clk` geldiği anda kalıcı olarak registerya yazılır ve PC güncellenir.

```
1 always_ff @(posedge clk or negedge rstn) begin
2     if (reg_write_en && rd != 0) begin
3         registers[rd] <= wdata; // x2 icine 5 yazilir.
4     end
5 end
```

Listing 4: Write-back (`register_file.sv`)

Bu iki işlemin aynı anda gerçekleşmesiyle komutun yaşam döngüsü tamamlanır ve işlemci `0x8000_0004` adresindeki yeni komutu işlemeye başlar.

2 Tasarımı ve Geliştirme Süreci

SystemVerilog, Vivado ve Verilator kullanarak Register File, ALU, Decoder ve bu modülleri bir araya getirdiğim top module'ü açıklamaya çalışacağım.

2.1 Register File (register Öbeği) Tasarımı

İşlecimin veri depolama ve okuma merkezi olan Register File, mem adında 32 adet 32-bitlik bellek kutucuğundan (logic [31:0] mem [0:31]) oluşacak şekilde tasarlanmıştır.

Tasarımda dikkat edilen en kritik iki donanım kuralı şunlardır:

1. **Asenkron Okuma, Senkron Yazma:** Veri okuma işlemi saat sinyalinden bağımsız (combinational) olarak anında gerçekleşirken, veri yazma işlemi veri güvenliği için sadece saat sinyalinin yükselen kenarında (posedge clk_i) gerçekleşir.
2. **Sıfırıncı register Kuralı:** RISC-V mimarisi gereği x0 registersı donanımsal olarak her zaman sıfıra sabitlenmiştir. Bu nedenle okuma asenkron atamalarında (raddr == 0) ve senkron yazma bloğunda (waddr_i != 0) koşulları eklenmiştir.

```
1 module regfile (  
2     input logic      clk_i,  
3     input logic      rstn_i,  
4  
5     // Okuma Portu 1 (rs1)  
6     input logic [4:0] raddr1_i,  
7     output logic [31:0] rdata1_o,  
8  
9     // Okuma Portu 2 (rs2)  
10    input logic [4:0] raddr2_i,  
11    output logic [31:0] rdata2_o,  
12  
13    // Yazma Portu (rd)  
14    input logic      we_i,          // Write Enable (Yazma Izni)  
15    input logic [4:0] waddr_i,  
16    input logic [31:0] wdata_i  
17 );  
18  
19    // 32 adet 32-bitlik hafıza dizisi yaratıyoruz  
20    logic [31:0] mem [0:31];  
21  
22    // -----  
23    // ASENKRON OKUMA (Kombinasyonel)  
24    // -----  
25    // RISC-V Kuralı: Eger adres 0 ise her zaman 0 dondur!  
26    assign rdata1_o = (raddr1_i == 5'b0) ? 32'b0 : mem[raddr1_i];  
27    assign rdata2_o = (raddr2_i == 5'b0) ? 32'b0 : mem[raddr2_i];  
28  
29    // -----  
30    // SENKRON YAZMA (Flip-Flop)  
31    // -----  
32    // Yazma islemi sadece saat vurusunda (posedge clk) gerçekleşir.  
33    always_ff @(posedge clk_i or negedge rstn_i) begin  
34        if (!rstn_i) begin  
35            // Reset gelince tum register'lari sıfırla  
36            for (int i = 1; i < 32; i++) begin  
37                mem[i] <= 32'b0;  
38            end  
39        end else begin  
40            // Yazma izni (we_i) varsa VE yazilacak adres x0 degilse yaz!  
41            if (we_i && waddr_i != 5'b0) begin  
42                mem[waddr_i] <= wdata_i;  
43            end  
44        end  
45    end  
46  
47 endmodule
```

Listing 5: Register File Tasarimi (regfile.sv)

2.2 ALU (Aritmetik Mantık Birimi) Tasarımı

İşleciminin hesaplama motoru olan ALU, kombinasyonel (saat sinyalinden bağımsız) bir devre olarak tasarlanmıştır. Bu tasarımda karmaşıklığı önlemek ve kod okunabilirliğini artırmak için klasik "3-bitlik kontrol sinyalleri" yerine, riscv_pkg::* paketinde tanımlanmış olan operation_e (Enum) yapısı kullanılmıştır.

Tasarımın öne çıkan yapısal özellikleri şunlardır:

- **Grup Mantığı:** Toplama işlemi gerektiren tüm komutlar (ADD, ADDI, LW, SW, JAL vb.) aynı case bloğunda birleştirilerek donanım israfı önlenmiştir.
- **İşaretli (Signed) İşlemler:** Aritmetik sağa kaydırma (SRA) ve büyüklük/küçüklük karşılaştırmalarında (SLT, BLT) SystemVerilog'un \$signed() fonksiyonu kullanılarak negatif sayıların işaret bitlerinin korunması sağlanmıştır.
- **Dallanma (Branch) Karar Mekanizması:** Sadece basit bir zero bayrağı üretmek yerine; ALU, gelen komut tipine göre tüm koşulları (BEQ, BNE, BGEU vb.) kendi içinde değerlendirerek doğrudan branch_taken_o sinyalini üretmektedir.

```
1 module alu (
2     input logic [31:0] a_i,           // 1. İşlem görececek sayı
3     input logic [31:0] b_i,           // 2. İşlem görececek sayı
4     input operation_e op_i,           // Decoder'dan gelen komut turu
5     output logic [31:0] result_o,      // 32-bit İşlem Sonucu
6     output logic branch_taken_o       // Branch şartı sağlandı mı?
7 );
8
9     always_comb begin
10         // Latch oluşmasını önlemek için varsayılan (default) değerler
11         result_o = 32'b0;
12         branch_taken_o = 1'b0;
13
14         case (op_i)
15             // Toplama İşlemleri (Veri, Adres ve PC hesaplamaları)
16             ADD, ADDI, LW, LH, LB, LHU, LBU, SW, SH, SB, AUIPC, JAL, JALR: begin
17                 result_o = a_i + b_i;
18             end
19
20             // Çıkarma ve Mantıksal İşlemler
21             SUB: result_o = a_i - b_i;
22             AND, ANDI: result_o = a_i & b_i;
23             OR, ORI: result_o = a_i | b_i;
24             XOR, XORI: result_o = a_i ^ b_i;
25
26             // Kaydırma (Shift) İşlemleri
27             SLL, SLLI: result_o = a_i << b_i[4:0];
28             SRL, SRLI: result_o = a_i >> b_i[4:0];
29             SRA, SRAI: result_o = $signed(a_i) >>> b_i[4:0];
30
31             // Karşılaştırmalar (Set Less Than) İşlemleri
32             SLT, SLTI: result_o = ($signed(a_i) < $signed(b_i)) ? 32'd1 : 32'd0;
33             SLTU, SLTIU: result_o = (a_i < b_i) ? 32'd1 : 32'd0;
34
35             LUI: result_o = b_i;
36
37             // Branch (Dallanma) Kontrolleri
38             BEQ: branch_taken_o = (a_i == b_i);
39             BNE: branch_taken_o = (a_i != b_i);
40             BLT: branch_taken_o = ($signed(a_i) < $signed(b_i));
41             BGE: branch_taken_o = ($signed(a_i) >= $signed(b_i));
42             BLTU: branch_taken_o = (a_i < b_i);
43             BGEU: branch_taken_o = (a_i >= b_i);
44
45             default: begin
46                 result_o = 32'b0;
47                 branch_taken_o = 1'b0;
48             end
49         endcase
50     end
51 endmodule
```

Listing 6: Geliştirilmiş ALU Tasarımı (alu.sv)

2.3 Top Module (riscv_singlecycle.sv): Veri Yolu ve Sistem Entegrasyonu

Sistemin kalbini oluşturan Top Module, sadece alt modülleri çağırmakla kalmaz; aynı zamanda bellek iletişimini (RAM/ROM), veri yolu yönlendirmelerini (Mux) ve Program Counter (PC) güncellemelerini yönetir. Bu karmaşık yapı, anlaşılabilirliği artırmak için işlemci boru hattı (pipeline) mantığına benzer şekilde 5 ana mantıksal bloğa ayrılarak tasarlanmıştır.

2.3.1 Blok 1: Komut Getirme (Fetch) ve PC Yönetimi

Sistem resetlendiğinde PC, RISC-V standartlarına uygun olarak 0x8000_0000 adresinden başlar. İşlemci, bu adresi kullanarak Instruction Memory’den (IMEM) 32 bitlik komutu çeker. Ayrıca bir sonraki adımda PC’nin 4 mü artacağı, yoksa bir Branch/Jump komutuyla başka bir adrese mi zıplayacağı (`next_pc`) bu blokta hesaplanır.

```
1 // Instruction Memory Okuma
2 assign instr = instr_mem[pc_reg[12:2]];
3
4 // Senkron PC Guncelleme
5 always_ff @(posedge clk_i or negedge rstn_i) begin
6     if (!rstn_i) pc_reg <= 32'h8000_0000;
7     else pc_reg <= next_pc;
8 end
9
10 // Asenkron Next PC Mantigi
11 always_comb begin
12     if (dinstr.is_jump) begin
13         next_pc = alu_result; // JAL ve JALR icin
14     end else if (dinstr.is_branch && branch_taken) begin
15         next_pc = pc_reg + dinstr.imm; // Dallanma sarti saglandiyisa
16     end else begin
17         next_pc = pc_reg + 4; // Normal isleyis
18     end
19 end
```

Listing 7: Fetch ve PC Guncelleme

2.3.2 Blok 2: Decoder ve Struct Kullanımı

Klasik tasarımlarda Decoder’dan çıkan onlarca sinyal karmaşaya yol açarken, bu tasarımda `dinstr_t` adında özel bir struct (yapı) kullanılmıştır. Komutun `opcode`, `funct3` ve `funct7` bitleri analiz edilerek; komutun geçerliliği, bellek/register kullanıp kullanmadığı ve ürettiği Immediate (sabit sayı) değeri bu paket içine işlenir.

```
1 assign opcode = instr[19:13]; // Custom ISA Opcode Kesimi
2
3 always_comb begin
4     // Varsayılan degerler ataniyor (Latch onlemek icin)
5     dinstr.valid = 1'b0; dinstr.is_store = 1'b0; // ...
6
7     case (opcode)
8         7'b1100100: begin // I-TYPE ALU Ornegi
9             dinstr.valid = 1'b1;
10            dinstr.rd_used = 1'b1;
11            dinstr.imm_used = 1'b1;
12            // Isaret uzatma (Sign-extension) yapiliyor
13            dinstr.imm = { {20{instr[31]}}, instr[31:20] };
14            case (funct3)
15                3'b000: dinstr.op = ADDI;
16                // Diger komutlar...
17            endcase
18        end
19        // Diger Opcode durumlar (R-Type, Load, Store, Branch vs.)
20    endcase
21 end
```

Listing 8: Decoder ve Struct Atamaları (Özet)

2.3.3 Blok 3: Veri Yolu Yönlendirmesi (MUX) ve Alt Modüller

Decoder'dan çıkan bilgilere göre, ALU'nun hangi verilerle işlem yapacağı seçilir (Multiplexer mantığı) ve daha önce tasarlanan regfile ile alu modülleri çağrılarak (instantiation) kablo bağlantıları yapılır.

```
1  always_comb begin
2      // ALU Giriş A: AUIPC ve JAL komutlarında PC, diğerlerinde rs1
3      if (dinstr.op == AUIPC || dinstr.op == JAL) alu_in_a = pc_reg;
4      else alu_in_a = rs1_data;
5
6      // ALU Giriş B: Branch/Register işlemlerinde rs2, diğerlerinde Immediate
7      if (dinstr.is_branch || (dinstr.valid && !dinstr.imm_used)) alu_in_b = rs2_data;
8      else alu_in_b = dinstr.imm;
9  end
10
11 // Modüllerin Bağlanması
12 assign reg_write_en = dinstr.valid & dinstr.rd_used;
13
14 regfile u_regfile ( /* Bağlantılar... */ );
15 alu u_alu ( /* Bağlantılar... */ );
```

Listing 9: ALU Girişleri ve Alt Modüllerin Çağırılması

2.3.4 Blok 4: Data Memory (RAM) ve Sub-word İşlemleri

En kritik tasarımlardan biri de bellek iletişimidir. İşlemci sadece 32-bit (Word) değil, 8-bit (Byte) veya 16-bit (Half-word) verileri de okuyup yazabilmelidir. ALU'dan gelen adresin son iki bitine (alu_result[1:0]) bakılarak veri içinde doğru kısım maskelenir ve işaret bitleri (sign-extension) korunarak registerya yazılacak nihai sonuç (reg_write_data) belirlenir.

```
1  // Data Memory Okuma (Kelime bazlı)
2  assign mem_read_data = data_mem[alu_result[12:2]];
3
4  // Alt-Kelime Okuma Maskeleyesi (Byte ve Half-Word)
5  assign read_byte = (alu_result[1:0] == 2'b00) ? mem_read_data[7:0] : /* diğerleri... */;
6  assign read_half = (alu_result[1] == 1'b0) ? mem_read_data[15:0] : mem_read_data[31:16];
7
8  // Registera Ne Yazılacak? (Write-Back Mux)
9  always_comb begin
10     if (dinstr.is_jump) begin
11         reg_write_data = pc_reg + 4; // JAL/JALR geri dönüş adresi
12     end else if (dinstr.is_mem && !dinstr.is_store) begin
13         case (dinstr.op)
14             LB: reg_write_data = { {24{read_byte[7]}}, read_byte }; // İşaret
15             // uzatmalı okuma
16             LW: reg_write_data = mem_read_data; // Tam kelime okuma
17             // Diğer Load komutları...
18         endcase
19     end else begin
20         reg_write_data = alu_result; // Normal ALU işlemi sonucu
21     end
22 end
```

Listing 10: Data Memory ve Write-Back MUX

2.3.5 Blok 5: Testbench Çıkış Portları

Simülasyonun (Vivado) ve Python doğrulama betiğinin işlemci içindeki durumları görebilmesi için, işlenen veriler işlemcinin dış pinlerine (output) aktarılır.

```
1  assign pc_o      = pc_reg;
2  assign instr_o   = instr;
3  assign update_o  = dinstr.valid;
4
5  // Eğer komut rd kullanmıyorsa Testbench yanıtılmamak için 0 gönderilir
6  assign reg_addr_o = dinstr.rd_used ? dinstr.rd_idx : 5'b0;
7  assign reg_data_o = reg_write_data;
8
```

```

9      assign mem_addr_o = alu_result;
10     assign mem_data_o = rs2_data;
11     assign mem_wrt_o  = dinstr.valid & dinstr.is_store;
12     assign data_o     = mem_read_data;

```

Listing 11: Dis Dunyaya Aktarım

3 Karşılaşılan Sorunlar ve Hata Ayıklama (Debug) Süreci

Bu ödev süreci, yalnızca bir donanım mimarisi tasarlamaktan ibaret olmamış; aynı zamanda simülasyon ortamlarının (Vivado) davranışlarını, testbench varsayımlarını ve işletim sistemi seviyesindeki kısıtlamaları anlamayı gerektiren yoğun bir hata ayıklama (debug) serüveni olmuştur. Aşağıda, projenin geliştirilmesi sırasında karşılaşılan hatalar, bu hataların donanımsal/yazılımsal analizleri ve uygulanan çözümler kronolojik olarak listelenmiştir.

3.1 Hata 1: Yarım Kalan Log Dosyası (Eksik Satırlar)

Sorun: `base_test` çalıştırıldığında oluşturulan `model.log` dosyası, standart olan `test.log` dosyasına (1940 satır) kıyasla sadece 431 satır içeriyor ve son satır karakter bazında yarım kalmış şekilde bitiyordu.

Analiz: Hatanın işlemcinin matematiksel bir hatasından değil, simülasyon süresinden kaynaklandığı tespit edilmiştir. Vivado'nun varsayılan simülasyon süresi 1000 ns'dir. Testbench dosyasındaki `$finish`; komutu ise #4000; (4000. ns) değerine ayarlanmıştır. Simülasyon 1000. ns'de durduğu için bellek (buffer) tam olarak diske yazılamamış ve dosya yarım kalmıştır.

Çözüm: Vivado simülasyon ekranında varsayılan "Run" (1000 ns) butonu yerine "Run All" (sonsuzluk işareti) butonuna basılarak simülasyonun testbench içindeki `$finish` komutunu görene kadar çalışması sağlanmıştır.

3.2 Hata 2: Başlangıç Adresi (Program Counter) Uyuşmazlığı

Sorun: Çıktılar karşılaştırıldığında, referans dosyada Program Counter (PC) değerleri `0x80000000`'dan başlarken, tasarlanan modelde `0x00000000`'dan başladığı görülmüştür.

Analiz: İşlemci, komutları `imem_data[pc_reg[12:2]]` üzerinden okuduğu için donanımsal olarak üst bitleri (8 rakamını) göz ardı etmiş ve doğru komutları çekerek işlemleri doğru yapmıştır. Ancak testbench, log dosyasına PC'nin 32 bitlik tam halini yazdırdığı için görsel bir eşleşme oluşmuştur. RISC-V mimarilerinde RAM/Instruction başlangıç adresi genelde `0x80000000` kabul edilir.

Çözüm: `riscv_singlecycle.sv` dosyasındaki PC reset bloğu güncellenmiştir:

```

1 // Eski Hal
2 if (!rstn_i) pc_reg <= '0;
3 // Yeni Hal (Düzeltilmiş)
4 if (!rstn_i) pc_reg <= 32'h8000_0000;

```

Listing 12: PC Reset Vektörü Duzeltmesi

3.3 Hata 3: Testbench Varsayımı ve Gereksiz Register Çıktıları

Sorun: Branch (BEQ, BNE) komutlarının test edildiği satırlarda, referans dosyada boşluk olması gerekirken, model çıktısında `x17 0x00000000` veya `x16 0x00000000` gibi fazladan register yazdırma işlemleri görülmüştür.

Analiz: Branch veya Store komutlarında hedef register (`rd`) kullanılmaz; bu bitler Immediate (sabit sayı) değerini taşır. Ancak Decoder modülü her ihtimale karşı `rd` bitlerini ayırıp çıkışa veriyordu. Testbench ise işlemciden "yazma izni" (`reg_write_en`) sinyali almadığı için, yalnızca adres kablosunun 0'dan farklı olup olmadığına bakarak log dosyasına sahte adresler basıyordu.

Çözüm: Bu sorunu çözmek için `riscv_pkg` paketindeki `dinstr_t` yapısında halihazırda bulunan ancak aktif kullanılmayan `rd_used` bayrağı devreye sokulmuştur. Decoder içerisinde komut türüne göre bu bayrak ayarlanmış (R-Type için 1, Branch/Store için 0) ve Top Module çıkışında testbench'i yönlendirecek bir "susturucu filtre" olarak kullanılmıştır:

```

1 // Eger komut (örneğin Branch) gerçekten bir register kullanmıyorsa (rd_used == 0),
2 // icerideki adres ne olursa olsun testbench'i yanıltmamak için 0 gönderilir.
3 assign reg_addr_o = dinstr.rd_used ? dinstr.rd_idx : 5'b0;

```

Listing 13: Testbench İçin Register Çikisi Maskeleye

3.4 Hata 4: Boş Log Dosyası ve "Buffer Flush" (Önbellek Boşaltma) Keşfi

Sorun: Hafıza iletişimini test eden `book_test` senaryosuna geçiş yapıp yeni `imem.mem` dosyası sisteme yüklendiğinde, `model.log` dosyasının inatla bomboş (0 KB) kaldığı görülmüştür. İşlemci dalga (waveform) ekranında kusursuz çalışıyor olmasına rağmen dosyaya hiçbir veri yazılmıyordu.

Analiz ve Çözüm: Deneme-yanılma süreci sırasında oldukça ilginç bir simülatör/yazılım davranışı keşfedilmiştir. Vivado programı tamamen kapatıldığında veya testbench içindeki çıktı dosyasının adı kod üzerinden değiştirilip (`$fopen("book_sonuc.log", "w");`) simülasyon yeni bir dosyaya yönlendirildiğinde; o ana kadar 0 KB olan eski `model.log` dosyasının bir anda 1 KB boyutuna ulaşarak verilerle dolduğu gözlemlenmiştir.

Bunun arka planındaki sebep şudur: `model.log` dosyası önceki test aşamasında VSCode gibi bir editörde açık unutulduğu için Windows işletim sistemi tarafından kilitlenmiş ve Vivado'nun dosyaya yazmasına izin verilmemiştir. Ancak Vivado çöküp hata vermek yerine, ürettiği tüm log verilerini inatla kendi önbelleğinde (RAM) biriktirmiştir. Biz Vivado'yu kapattığımızda veya yeni bir dosyaya yönelip eskisinin yakasını bıraktığımızda ise, içinde biriken tüm bu verileri aniden eski dosyaya kusmuştur (Buffer Flush).

Sonuç: Karşılaşılan sorun işlemci mimarisinden değil, tamamen editör/işletim sisteminin dosya kitleme mekanizması ve simülatörün önbellek davranışı kaynaklı çıkmıştır. Aniden güncellenen ve verilerle dolan bu doğru `model.log` dosyası Python doğrulama betiğine (`check_result.py`) sokulmuş ve tüm satırlarda **PASS** sonucu alınarak proje %100 başarıyla tamamlanmıştır.

3.5 Hata 5: Verilator Coverage Hatası ve C++ Arka Kapı Backdoor Çözümü

Sorun: Kapsam raporu (HTML) oluşturmak için `make cov` komutu çalıştırıldığında simülatör hata vererek çökmüştür. Hatanın kaynağı, Makefile dosyasındaki `+verilator+coverage+file+logs/coverage.dat` argümanının sistemdeki Verilator sürümü tarafından tanınmamasıdır ("Unknown runtime argument" hatası).

Analiz ve Çözüm: Araç (tool) sürümünden kaynaklanan bu sorunu aşmak için iki aşamalı bir yöntem izlenmiştir:

1. Makefile Düzenlemesi: Hata veren `+verilator...` kısmı, raporun kaydedileceği dizini ve dosya adını belirten özel bir parametredir. Mevcut Verilator sürümü bu uzun parametreyi desteklemediği için satırdan tamamen silinmiş ve komutun sadece ana çalıştırıcı kısmı (`obj_dir/Vtb`) bırakılmıştır. Bu işlem simülatörün çökmesini engellemiştir.

2. C++ Backdoor Müdahalesi: Makefile argümanı silindiğinde simülasyon hatasız tamamlanmış, ancak bu kez de rapor dosyası diske yazılmamıştır. Bunu çözmek için SystemVerilog'un doğrudan C++ kodu çalıştırma özelliği (`$c`) kullanılmıştır. Testbench (`tb.sv`) dosyasında, `$finish` komutundan hemen önce Verilator'ın iç kütüphanesine raporu zorla diske yazdırması için şu satır eklenmiştir:

```
1 // Simulasyon kapanmadan hemen önce raporu diske yazma emri (Backdoor)
2 $c("VerilatedCov::write(\"coverage.dat\");");
3 $finish;
4 end
```

Listing 14: Testbench Icerisinden Verilator C++ Kutuphanesine Mudahale

Sonuç: Makefile içerisindeki argüman temizliği ve testbench içerisine eklenen C++ müdahalesi sayesinde simülatör kısıtlamaları aşılmış; %100 kapsam oranına (Full Coverage) sahip HTML raporu başarıyla üretilmiştir.

Kaynakça

- [1] Mano, M. M., & Ciletti, M. D. *Digital Design: With an Introduction to the Verilog HDL* (5th ed.). Pearson.
- [2] Harris, D. M., & Harris, S. L. *Digital Design and Computer Architecture*. Morgan Kaufmann.
- [3] Patterson, D. A., & Hennessy, J. L. *Computer Organization and Design: The Hardware/Software Interface* (5th ed.). Morgan Kaufmann.
- [4] MTH401E Dersi Sunum Slaytları ve Ders Notları.
- [5] AMD Adaptive Computing Documentation (Vivado Design Suite User Guides). *Advanced Micro Devices, Inc.*

• Bu projenin kodlama sürecini hızlandırmak, temel donanım mimarisi, mantıksal tasarımı ve hata ayıklama (debug) süreçlerini daha detaylı anlayabilmek *case* ve *ternary* gibi SystemVerilog yapılarının syntax kontrollerini sağlamak, komut satırı işlemlerini düzenlemek ve hazırlanan taslak metinleri akademik bir dille toparlayarak LaTeX formatına dökmek amacıyla Google Gemini yapay zeka asistanından destek alınmıştır.